

# Smart Asset Management and Resource Allocation Platform

## Design Document

Submitted for: Cult Open Projects 2026 — IIT Roorkee Cultural Council

Track: Software Development

Live Demo (Frontend): <https://asset-management-project-71ed.vercel.app/>

Live Demo (Backend API): <https://asset-management-project-bmlf.onrender.com>

GitHub Repository: <https://github.com/himanshuchaudhari969/asset-management-project>

# 1. Problem Understanding

Cultural sections under the Cultural Council manage a large pool of shared resources such as DSLR cameras, audio systems, lighting equipment, costumes, props and event infrastructure. Currently this is tracked through informal channels like spreadsheets, registers and group chats, which leads to scheduling conflicts, underutilization, delayed returns and lack of accountability.

The goal of this project is to build a centralized, role-based web platform that allows administrators to manage an inventory of assets, and lets users discover, search and book those assets for specific durations. Admins can approve or reject requests, track issued and returned items, and view analytics on usage patterns. The platform also supports in-app notifications, an audit trail of admin actions, and QR-code generation for quick asset identification.

## 1.1 Goals

- Provide a single source of truth for asset inventory and availability.
- Allow users to search/filter assets and raise booking requests for a date range.
- Give admins an approval workflow (approve / reject / mark returned).
- Maintain accurate available-quantity counts throughout the asset lifecycle.
- Provide an analytics dashboard for utilization, booking trends and overdue returns.
- Maintain an audit log of administrative actions for accountability.
- Notify users in-app when their booking status changes.

## 2. System Architecture

The application follows a standard three-tier client-server architecture with a React single-page application on the frontend, a REST API built with Node.js and Express on the backend, and a PostgreSQL database for persistence. The frontend communicates with the backend over HTTPS using JSON, and authenticates using JWT tokens stored in the browser's local storage.

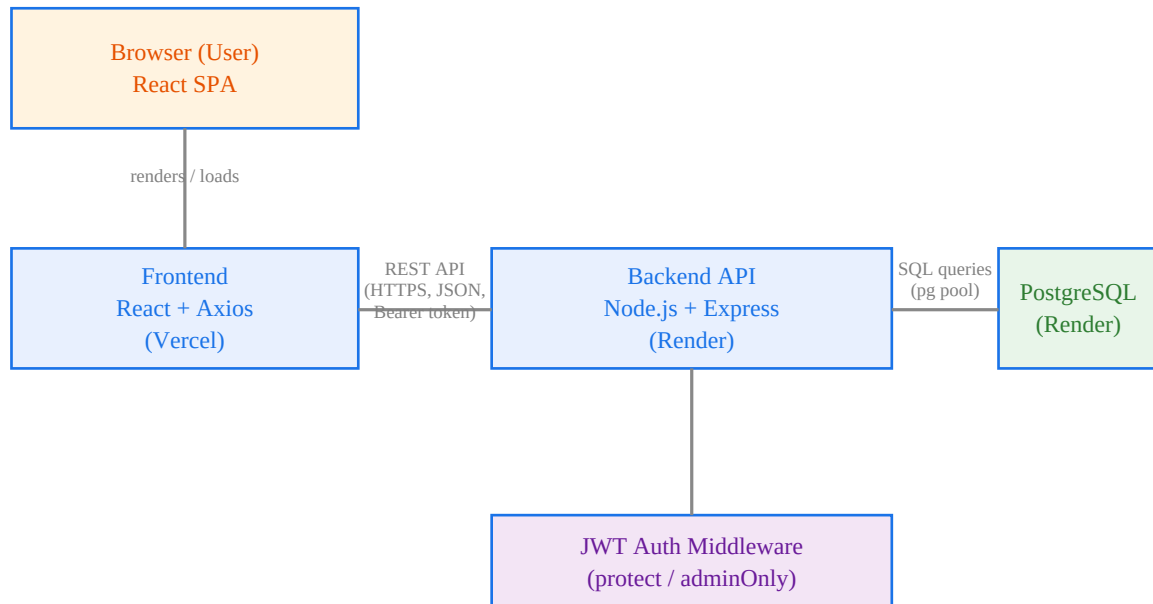


Figure 1: High-level system architecture

### 2.1 Layers

**Presentation Layer:** React app (Vercel) — Login/Register, Dashboard, Assets, Bookings, Analytics and Audit Logs pages. Uses React Router for navigation and Axios for API calls.

**Application Layer:** Express REST API (Render) organized into routes → controllers. Handles authentication, validation, business logic for bookings/inventory, analytics aggregation, audit logging and notifications.

**Data Layer:** PostgreSQL database (Render) storing users, assets, bookings, audit\_logs and notifications tables, accessed via the 'pg' connection pool.

### 2.2 Authentication & Authorization

Users register/login via /api/auth endpoints. Passwords are hashed using bcrypt before being stored. On successful login, the server issues a JWT containing the user's id and role. The frontend stores this token in localStorage and attaches it as a Bearer token on every subsequent request. Two middleware functions guard routes: **protect** verifies the JWT and attaches the user to the request, while **adminOnly** additionally checks that the user's role is 'admin' before allowing access (e.g. asset creation, approvals, analytics, audit logs).

### 3. Database Schema & ERD

The schema consists of five tables. 'bookings' references both 'users' and 'assets'; 'audit\_logs' and 'notifications' both reference 'users'.



Figure 2: Entity Relationship Diagram

### 3.1 Table Details

#### users

| Column     | Type         | Notes                             |
|------------|--------------|-----------------------------------|
| id         | SERIAL PK    | Auto increment                    |
| name       | VARCHAR(100) | Required                          |
| email      | VARCHAR(100) | Unique, required                  |
| password   | VARCHAR(255) | bcrypt hash                       |
| role       | VARCHAR(10)  | 'user' or 'admin', default 'user' |
| created_at | TIMESTAMP    | Default NOW()                     |

#### assets

| Column      | Type         | Notes                           |
|-------------|--------------|---------------------------------|
| id          | SERIAL PK    | Auto increment                  |
| name        | VARCHAR(100) | Required                        |
| category    | VARCHAR(50)  | e.g. Camera, Audio, Props       |
| quantity    | INTEGER      | Total owned, default 1          |
| available   | INTEGER      | Currently free units, default 1 |
| description | TEXT         | Optional                        |
| created_at  | TIMESTAMP    | Default NOW()                   |

#### bookings

| Column     | Type        | Notes                                    |
|------------|-------------|------------------------------------------|
| id         | SERIAL PK   | Auto increment                           |
| user_id    | INTEGER FK  | References users(id)                     |
| asset_id   | INTEGER FK  | References assets(id)                    |
| start_date | DATE        | Required                                 |
| end_date   | DATE        | Required                                 |
| status     | VARCHAR(20) | pending / approved / rejected / returned |
| reason     | TEXT        | Optional note from user                  |
| created_at | TIMESTAMP   | Default NOW()                            |

#### audit\_logs & notifications

| Table      | Column                                   | Notes                                                                             |
|------------|------------------------------------------|-----------------------------------------------------------------------------------|
| audit_logs | id, user_id, action, details, created_at | Records admin actions, e.g. asset created/updated/deleted, booking status changes |

| Table         | Column                                    | Notes                                                           |
|---------------|-------------------------------------------|-----------------------------------------------------------------|
| notifications | id, user_id, message, is_read, created_at | In-app messages sent to users on booking updates / new requests |

## 4. API Overview

Base URL (production): <https://asset-management-project-bmlf.onrender.com>

| Method | Endpoint                 | Description                                  | Access    |
|--------|--------------------------|----------------------------------------------|-----------|
| POST   | /api/auth/register       | Register a new user                          | Public    |
| POST   | /api/auth/login          | Login, returns JWT + user                    | Public    |
| GET    | /api/assets              | List assets (supports ?search= & ?category=) | Protected |
| POST   | /api/assets              | Create a new asset                           | Admin     |
| PUT    | /api/assets/:id          | Update an asset                              | Admin     |
| DELETE | /api/assets/:id          | Delete an asset                              | Admin     |
| POST   | /api/bookings            | Create a booking request                     | Protected |
| GET    | /api/bookings/my         | Get current user's bookings                  | Protected |
| GET    | /api/bookings/all        | Get all bookings                             | Admin     |
| PUT    | /api/bookings/:id/status | Approve / reject / mark returned             | Admin     |
| GET    | /api/analytics           | Aggregated usage & utilization data          | Admin     |
| GET    | /api/notifications       | Get current user's notifications             | Protected |
| PUT    | /api/notifications/read  | Mark all notifications as read               | Protected |
| GET    | /api/audit               | Get last 100 audit log entries               | Admin     |

### 4.1 Sample Flow — Booking an Asset

1. User logs in and receives a JWT token.
2. User browses GET /api/assets, optionally filtering by search/category.
3. User submits POST /api/bookings with asset\_id, start\_date, end\_date and an optional reason.
4. Server checks the asset's 'available' count; if zero, the request is rejected.
5. On success, a row is inserted into 'bookings' with status 'pending', an audit log entry (BOOKING\_CREATED) is written, and all admins receive a notification.
6. Admin reviews the request via GET /api/bookings/all and calls PUT /api/bookings/:id/status with 'approved' or 'rejected'.
7. If approved, the asset's 'available' count is decremented by 1. If later marked 'returned', it is incremented back. The user is notified of the status change.

## 5. Design Decisions

### Why JWT over sessions

JWT is stateless and works well with a decoupled frontend/backend deployed on separate platforms (Vercel and Render). It avoids the need for server-side session storage and scales easily.

### Single 'bookings' table with a status field

Rather than separate tables for requests, approvals and returns, a single status field (pending/approved/rejected/returned) keeps the lifecycle simple to query and reason about, while still allowing the full history to be viewed.

### Available count maintained on the assets table

Instead of computing availability on every read by aggregating active bookings, the 'available' column is updated incrementally on approval/return. This keeps booking lookups (which double-check availability) fast and simple, at the cost of needing consistent updates inside the approval/return logic.

### Centralized audit logging via a helper function

A small 'logAction(userId, action, details)' helper is called from controllers (asset create/update/delete, booking status changes) so that all administrative actions are recorded consistently without duplicating SQL across controllers.

### In-app notifications instead of email

Given the scope and timeline, in-app notifications (stored in a 'notifications' table and surfaced via a bell icon) provide immediate feedback to users without requiring email service configuration, while still satisfying the optional notification requirement.

### QR codes generated client-side

QR codes are generated on the fly in the browser using 'qrcode.react', encoding the asset's id, name and category. This avoids storing image files on the server and keeps the QR content always in sync with the latest asset data.

### Search & filter implemented server-side

The /api/assets endpoint accepts optional 'search' and 'category' query parameters and builds a parameterized SQL query (ILIKE for search, exact match for category). This keeps filtering correct even as the inventory grows, rather than filtering a potentially large list on the client.

### Deployment split across three free-tier services

The frontend is deployed on Vercel, the backend (Express API) on Render as a Web Service, and PostgreSQL on Render's free database tier. This mirrors a realistic decoupled deployment while staying within free hosting limits.

## 5.1 Future Improvements

- Email notifications in addition to in-app notifications.
- Asset health/condition tracking and maintenance history.
- Admin-side QR scanning for issue/return at the point of handover.
- Pagination on assets/bookings/audit log endpoints for larger datasets.
- Dockerized local development setup.